f12a9704-ffd6-452a-bc15-ceb16571cad2

<current>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'github/spec-kit' repository.

<repository_context>
I've cloned the repository in the directory /tmp/workspace/github/spec-kit (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-05-14T17:42:02.819+00:00</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Extract agent context updates into bundled agent-context extension
</pr_title>

<pr_description>
New Feature

Coding agent context file management (`<!-- SPECKIT START -->` / `<!-- SPECKIT END -->` injection into `CLAUDE.md`, `.github/copilot-instructions.md`, etc.) was hardcoded into `IntegrationBase`, with no way to opt out or customize markers. This change moves that behavior into a bundled `agent-context` extension driven by `.specify/init-options.json`.

````
### What does this feature do?

Adds `extensions/agent-context/` (`bundled: true`, opt-out) that
owns the lifecycle of the managed context section. Both the
Python paths and the new shell/PowerShell scripts read
`context_file` and `context_markers` from `.specify/init-
options.json` — single source of truth, no per-agent logic,
user-customizable markers.

```json
{
  "context_file": "CLAUDE.md",
  "context_markers": {
    "start": "<!-- SPECKIT START -->",
    "end": "<!-- SPECKIT END -->"
  }
}
```

### Implementation details

- **`integrations/base.py`**
  - `_resolve_context_markers(project_root)` — reads
`context_markers` from `init-options.json`, falls back to
`CONTEXT_MARKER_START` / `CONTEXT_MARKER_END` constants per
side.
  - `_agent_context_extension_enabled(project_root)` — reads
`.specify/extensions/.registry`; returns `True` when registry/
entry absent (backwards compat) or `enabled != false`.
  - `upsert_context_section()` / `remove_context_section()` now
use the resolved markers and short-circuit when the extension is
disabled.

- **`specify_cli/__init__.py`**
  - `specify init` seeds `context_markers` defaults into
`init_opts` from the class constants.
  - `_update_init_options_for_integration()` seeds defaults
while preserving any user-customized markers.
  - `_clear_init_options_for_integration()` pops
`context_markers` alongside `context_file`.
  - New tracker step auto-installs the bundled extension during
`specify init` (parallel to the `git` extension flow).

- **`extensions/agent-context/`** — `extension.yml` (id `agent-
context`, hooks `after_specify` / `after_plan`), `README.md`,
`commands/speckit.agent-context.update.md`, and bash +
PowerShell `update-agent-context` scripts. Both scripts parse
````

`init-options.json`, resolve markers (with default fallback), auto-detect the most recent `specs/*/plan.md` when no plan path is supplied, and perform the same upsert algorithm as the Python path (CRLF normalization, BOM strip, marker-corruption recovery).

- **`extensions/catalog.json`** — `agent-context` registered as `bundled: true`, alphabetized first.

- **`AGENTS.md`** — wrapper-script and dispatcher-script guidance removed; replaced with the `init-options.json` flow and the `specify extension disable agent-context` opt-out.

- **`tests/extensions/test_extension_agent_context.py`** — 25 tests covering extension layout, catalog entry, marker resolution (defaults / custom / partial / invalid), upsert+remove with custom markers, the disabled-extension gate (`upsert` returns `None`, `remove` returns `False` and leaves file untouched), and `init-options.json` writers.

Backwards compatibility

- Missing `init-options.json` or missing `context_markers` → class-constant defaults, identical to current behavior.
- Missing extension registry or missing `agent-context` entry → treated as enabled, so existing projects continue to receive context updates without re-init.
- All 30+ existing integrations are unchanged; markers remain defined only on `IntegrationBase`.
</pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

</pr_comments>

<pr_reviews>

<comment_thread>
<file>src/specify_cli/integrations/base.py:582</file>
<original_commit>2e822be</original_commit>

```
</comment_thread>

<comment_thread>
<file>src/specify_cli/integrations/base.py:660-661</file>
<original_commit>2e822be</original_commit>



</comment_thread>

<comment_thread>
<file>extensions/agent-context/scripts/powershell/update-agent-
context.ps1:77-81</file>
<original_commit>2e822be</original_commit>



</comment_thread>

<comment_thread>
<file>src/specify_cli/__init__.py:976-988</file>
<original_commit>2e822be</original_commit>



</comment_thread>

<comment_thread>
<file>extensions/agent-context/scripts/powershell/update-agent-
context.ps1:51-103</file>
<original_commit>2e822be</original_commit>



</comment_thread>

<comment_thread>
<file>extensions/agent-context/scripts/bash/update-agent-
context.sh:29-74</file>
<original_commit>2e822be</original_commit>



</comment_thread>
```

<comment_thread>
<file>AGENTS.md:402</file>
<original_commit>2e822be</original_commit>

</comment_thread>

<comment_thread>
<file>src/specify_cli/__init__.py:996-1017</file>
<original_commit>2e822be</original_commit>

</comment_thread>

</pr_reviews>

<pr_comments>

<comment_new>
<comment_id>4450836415</comment_id>
<author>@mnriem</author>
@copilot apply changes based on the comments in <a
href="https://github.com/github/spec-kit/pull/
2546#pullrequestreview-4290062952">this thread
</comment_new>

</pr_comments>

<pr_reviews>

<comment_thread>
<file>src/specify_cli/__init__.py:984-988</file>
<original_commit>84df982</original_commit>

</comment_thread>

<comment_thread>
<file>extensions/agent-context/scripts/bash/update-agent-
context.sh:44-75</file>
<original_commit>84df982</original_commit>

isleading — the user has a valid extension config but the script silently no-ops because of an unmet runtime dependency. Consider distinguishing "PyYAML import failure" from "config missing context_file" and emitting an explicit diagnostic (or attempting a minimal fallback parser for the simple YAML this file uses), so a missing optional dependency does not appear as a configuration error. The same concern applies to `extensions/agent-context/scripts/powershell/update-agent-context.ps1` (Python fallback path imports `yaml` and silently returns `{}` on failure).
</comment_old>

</comment_thread>

<comment_thread>
<file>src/specify_cli/__init__.py:1018-1031</file>
<original_commit>84df982</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
The `finally` block unconditionally rewrites `context_file` in the extension config on every `specify init` (including reinit on a project where the extension was "already installed"). This means that even though `agent-context-config.yml` is presented to users as "edit to override" (see `extensions/agent-context/agent-context-config.yml` line 8), any user-edited `context_file` value is silently reset to the active integration's class default on every reinit. Markers are preserved (good), but the asymmetry between `context_file` (always overwritten) and `context_markers` (preserved) is undocumented — consider either (a) preserving a non-empty user-set `context_file` when it differs from `integration.context_file`, or (b) updating the config file's inline comment to make clear that `context_file` is owned by Spec Kit and will be regenerated on every init/integration switch.
</comment_old>

</comment_thread>

<comment_thread>
<file>extensions/agent-context/scripts/powershell/update-agent-context.ps1:49-103</file>
<original_commit>84df982</original_commit>

<comment_old>

<author>@copilot-pull-request-reviewer</author>
The PowerShell fallback path uses `ConvertFrom-Json -AsHashtable`, which is only available in PowerShell 6.0+. On Windows PowerShell 5.1 (the default on many Windows systems), `-AsHashtable` is not recognized and the `ConvertFrom-Json` call will fail, dropping into the outer catch and exiting with "unable to parse" — a silent no-op. Combined with the fact that `ConvertFrom-Yaml` is also unavailable on stock Windows PowerShell 5.1 (requires the `powershell-yaml` module), users on Windows PowerShell 5.1 without that module will see the agent context file silently never updated. Consider documenting the PowerShell version requirement, or implementing a minimal hand-rolled parser for the simple two-key YAML this file uses.
</comment_old>

</comment_thread>

<comment_thread>
<file>src/specify_cli/__init__.py:1551-1567</file>
<original_commit>84df982</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
`_clear_init_options_for_integration` only clears the agent-context extension's `context_file` when `opts["integration"]` or `opts["ai"]` matches `integration_key`. If neither matches but the legacy `context_file`/`context_markers` keys are present in `init-options.json` (e.g., from a project initialized on a prior version), they will be left in the file uncleaned. The cleanup of legacy keys (lines 1557–1558) is inside the `if` block, so it only runs when the integration matches. Consider moving the legacy-key cleanup out of the conditional so any project touched by the new code drops the now-unused fields from `init-options.json`.
</comment_old>

</comment_thread>

</pr_reviews>

<pr_comments>

<comment_new>
<comment_id>4453081071</comment_id>
<author>@mnriem</author>
@copilot apply changes based on the comments in <a href="https://github.com/github/spec-kit/pull/

2546#pullrequestreview-4290135196">this thread
</comment_new>

<comment_old>
<author>@copilot</author>

> @copilot apply changes based on the comments in <a
href="https://github.com/github/spec-kit/pull/
2546#pullrequestreview-4290135196">this thread

Applied the actionable fixes from that review thread in commit
`22d6ef5`.
</comment_old>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
Copilot's findings

- **Files reviewed:** 19/19 changed files
- **Comments generated:** 4

</comment_old>

</pr_comments>

<pr_reviews>

<comment_thread>
<file>src/specify_cli/agents.py:386</file>
<original_commit>22d6ef5</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
The new `__CONTEXT_FILE__` resolution path uses a bare `except
Exception: pass`, which silently swallows every error from
`_load_agent_context_config` (e.g., a corrupted YAML file, an
unexpected import-time failure, etc.). This is exactly the
pattern that previous review feedback asked to avoid in
`IntegrationBase._resolve_context_markers` (which now narrowly
catches `ImportError` / `OSError` / `ValueError`). For
consistency and to avoid masking real bugs, narrow the except to
the specific exceptions you actually want to tolerate (e.g.,
`ImportError`, `OSError`, `yaml.YAMLError`).

</comment_old>

</comment_thread>

<comment_thread>
<file>extensions/agent-context/scripts/bash/update-agent-context.sh:44-80</file>
<original_commit>22d6ef5</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
The bash script hard-requires Python 3 with PyYAML at runtime (the heredoc imports `yaml` and exits with code 2 on `ImportError`). This is a new external runtime dependency for projects using the `agent-context` extension. The prior shared `update-agent-context.sh` was pure shell. Two consequences worth flagging:
1. The shell `if ! _raw_opts="$(...)"` relies on Python's exit code: when PyYAML is missing, the heredoc prints to stderr and exits 2 — the script then prints "failed to read extension config; skipping update." with `exit 0`, so context updates silently stop happening on systems without PyYAML. Consider surfacing a clearer one-time message ("install PyYAML to re-enable agent-context updates") or documenting the dependency more prominently in the extension README (currently only mentioned as a brief "runtime note").
2. PyYAML is installed alongside the `specify` CLI, but plain `python3` invoked from a hook may use a different interpreter (system Python) where PyYAML is not available. Worth noting in the README troubleshooting section.
</comment_old>

</comment_thread>

<comment_thread>
<file>src/specify_cli/__init__.py:984-988</file>
<original_commit>22d6ef5</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
The PR description (in "What does this feature do?") still describes `context_file` and `context_markers` as living in `.specify/init-options.json`, but per the implementation and the resolution of earlier review feedback, both fields now live exclusively in `.specify/extensions/agent-context/agent-context-config.yml` (and the new `_clear_init_options_for_integration` /

`_update_init_options_for_integration` helpers actively *remove* these keys from `init-options.json` if a legacy version wrote them). Update the PR description's JSON example and the surrounding text so future readers don't confuse the location of the source of truth.
</comment_old>

</comment_thread>

<comment_thread>
<file>src/specify_cli/__init__.py:1018-1031</file>
<original_commit>22d6ef5</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
When the install step fails (the `except Exception` branch), the `finally` block still writes the agent-context extension config to `.specify/extensions/agent-context/agent-context-config.yml` via `_save_agent_context_config` (which `mkdir -p`s the parent). This means a project where the extension install failed will end up with the config file present but **no registry entry** for `agent-context`. As a side effect:
- `_agent_context_extension_enabled()` treats a missing registry entry as "enabled" (per the docstring at base.py:495-499), so context-section management still runs against an extension that the user effectively does not have installed.
- The bundled `commands/speckit.agent-context.update.md` and the bash/PowerShell scripts under `.specify/extensions/agent-context/scripts/` will be missing, but the Python upsert path will still mutate the agent context file.

This may be intentional (graceful degradation), but it's worth surfacing in the tracker error message so the user understands that the agent-context Python plumbing is active even though the extension install failed.
</comment_old>

</comment_thread>

</pr_reviews>

<pr_comments>

<comment_new>
<comment_id>4453198343</comment_id>
<author>@mnriem</author>
@copilot apply changes based on the comments in <a href="https://github.com/github/spec-kit/pull/

2546#pullrequestreview-4292120017">this thread
</comment_new>

</pr_comments>

</comments>

The last **11** git commits in this branch are the changes you have made so far. Use those as your change commit history.
</problem_statement><rules_for_making_changes>
* Start by fully understanding <problem_statement> and the <comments> before making any changes. You will only address the <comment_new> comments. Reply to new comments as you address them. Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a suggestion.
    - Is it addressed to you or someone else, e.g. "Hey @username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

<system_reminder>

</system_reminder>

A **CI/Build Failure** has been mentioned in the problem statement, make sure to follow your instructions for addressing CI/Build Failures.